

AI AGENT BASED SW DEVELOPMENT

같은 도구, 다른 결과

AI Agent 기반 SW 개발의 비법

84% 개발자가 AI를 씁니다 | 그런데 결과는 **3배** 넘게 벌어집니다

지금 코딩에 쓰는 모델

ANTHROPIC 에이전틱 코딩 · Claude Code

기본값 Opus 5.5 장기 실행 에이전틱 코딩 \$4 · \$20 입력 · 출력 / 1M 컨텍스트1M 기본 effortmedium 레포 전체 작업 다단계 구현 도구 사용	Fable 5.1 고난도 추론 · 최장기 작업 \$10 · \$50 입력 · 출력 / 1M 컨텍스트1M 기본 efforthigh 난이도 최상 설계·디버깅	Sonnet 5 속도와 지능의 균형 \$2 · \$10 입력 · 출력 / 1M 컨텍스트1M 기본 efforthigh 일상 개발 실습·반복 작업
--	---	---

OPENAI Codex

SW 엔지니어링 GPT-6 Astra 에이전틱 코딩 · 컴퓨터 사용 \$10 · \$50 입력 · 출력 / 1M 컨텍스트1.05M 출시2026.09 장기 자율 작업 컴퓨터 조작	GPT-6 Sol / Luna 비용 최적화 라인 \$2 · \$10 Sol — Luna \$0.1 · \$0.5 컨텍스트1.05M 출시2026.09 대량 처리 저비용 반복
---	---

가격 차이는 25배. 작업 난이도에 모델을 맞추는 것이 첫 번째 절약이다.

오픈 LLM 모델

— 전부 MoE — 총 파라미터는 메모리, 활성 파라미터는 연산량

XIAOMI · 중국 MiMo-V2.6 Pro 1.0T 총 파라미터 활성42B 지능 지수46 라이선스확인 필요 오픈 최상위 2026 하반기 오픈웨이트 지능 지수 최상위	Z AI · 중국 GLM-5.3 753B 총 파라미터 활성40B 지능 지수45 라이선스퍼미시브 장기 에이전틱터미널 작업 2026.08 장기 에이전틱 코딩에 가장 많이 추천	MOONSHOT AI · 중국 Kimi K3 2.8T 총 파라미터 활성104B 지능 지수44 라이선스Modified MIT 프론트엔드 강점 2026.07 프론트엔드 코딩에서 오픈 모델 첫 선두권	DEEPSEEK · 중국 V4 Pro 1.6T 총 파라미터 활성49B 컨텍스트1M 라이선스MIT 자체 호스팅저가 변종 2026.04 SWE-bench 오픈 선두 · 실제 버그 수정	ALIBABA · 중국 Qwen3 Coder 480B 총 파라미터 활성35B Next 계열활성 3B 라이선스Apache-2.0 로컬 구동효율 최상 2025.07 ~ 2026 활성 3B로 SWE-bench 70%대
--	--	---	---	---

상용 프론티어 54~58 · 오픈 44~46. 그 차이를 감수하고 얻는 것은 데이터를 내보내지 않는 선택지.

지능 지수: Artificial Analysis v4.3.2
라이선스는 도입 전 원문 확인

무엇을 재는 벤치마크인가

	01 SWE-bench Pro	02 Terminal-Bench	03 DeepSWE	04 LiveCodeBench
측정 관점	결과물	행위	장기 개발	인지
답하는 질문	우리 버그를 말길 수 있나	끝까지 스스로 갈 수 있나	큰 변경을 말길 수 있나	애초에 코드를 아나
입력 → 요구	저장소 + 이슈 본문 → 패치 생성	격리 컨테이너 + 지시 → 셀 조작	짧은 작업 설명 → 다중 파일 변경	알고리즘 문제 → 작성·예측
검증하는 것	- 결함 위치 파악 — 수만 줄 중 어디인지 - 요구 해석 — 이슈 글에서 의도 복원 - 회귀 방지 — 고치며 부수지 않기 - 최소 변경 — 손덜수록 불리	- 오류에서 복구 — 실패 출력 읽고 전환 - 장기 실행 유지 — 목표를 잊지 않기 - 도구 사용 판단 — 명령 선택과 순서 - 비용 효율 — 시도 횟수가 곧 돈	- 대규모 변경 완주 — 참조 해법이 5.5배 분량 - 자율 탐색 — 설명 한 줄만 주고 말긴다 - 기능 검증 — 구현 방식은 자유 - 오염 없음 — 비공개 신규 과제	- 코드 읽기 — 실행 없이 출력 예측 - 알고리즘 설계 — 처음부터 작성 - 자기 수정 — 실패한 코드 고치기 - 오염 저항 — 출제일로 걸러내기
검증 안 함	코드 품질 · 설계 타당성	누적된 코드베이스 맥락	Java · 표본 113개의 한계	실무 코드 · 레포 탐색
규모 · 상위	1,865 과제 · 약 23%	에이전트 단위 측정	113 과제 · 12 ~ 70%	계속 추가 · 미포화

순위가 서로 다른 것이 정상이다. 재는 대상이 다르기 때문.

지금도 만들어지는 중이다

01 모델은 아직 개발되고 있다

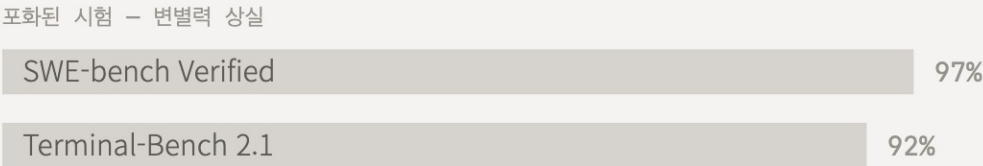
2026년 주요 출시



9개월에 프론티어 모델 **10개 이상**. 평균 **한 달에 한 번** 이상 판이 바뀐다.
Google Gemini 4는 아직 학습 중 — **끝난 경쟁이 아니다**.

모델도 벤치마크도 확정되지 않았다. 고정된 정답이 없으니 바뀌어도 살아남는 것에 투자한다 — 하네스와 루프.

02 벤치마크도 어려워진다



최고 점수 · 세트와 하네스에 따라 달라짐

점수가 90%대에 몰리면 **시험을 바꾼다**.
새 시험의 최고 점수는 다시 **23~70%**로 떨어진다.

출시 시점은 공개 자료 기준 근사치
점수는 측정 주체 · 세트별 상이

대화 상대에서 동료로

발전 방향

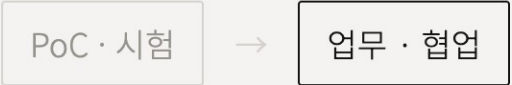
제어 흐름이 사람에서 모델로 넘어간다

01 자동완성 사람 커서 위치가 곧 지시 사람의 일 계속 타이핑	02 대화 사람 매 턴마다 지시 사람의 일 붙여넣고 옮기기	03 워크플로 사람 경로를 미리 고정 사람의 일 파이프라인 설계	04 에이전트 모델 경로를 정하고 스스로 검증 사람의 일 기준을 주고 결과를 검증
---	--	---	---

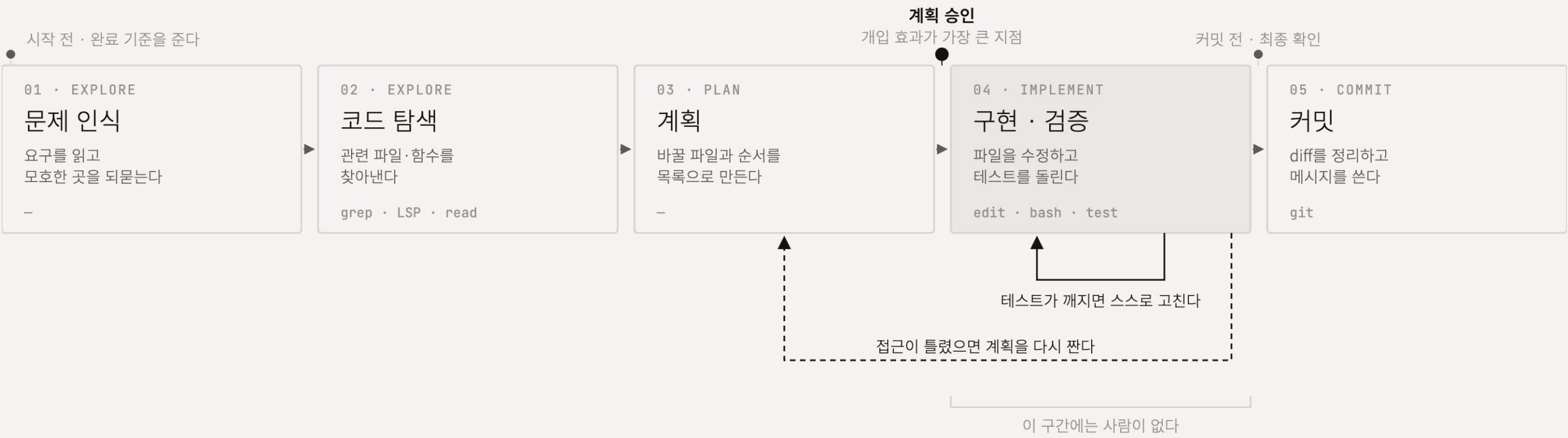
사람이 하는 일은 **실행**에서 → **설계와 판단**으로 옮겨간다

WHAT CHANGED

채팅창에 붙여넣어 **가능성을 시험하던 단계**를 지나,
이제는 실제 업무를 **맡기고 함께 수행하는 단계**로 넘어왔다.



한 번의 요청, 다섯 개의 단계



CORE 되돌아가는 화살표가 있으면 **에이전트**, 없으면 **자동완성**이다. Anthropic 공식 문서 참고
Explore-Plan-Implement-Commit Workflow

에이전트는 이 단계를 스스로 돈다.
우리가 설계하는 것은 단계가 아니라, **멈추는 조건과 되돌아가는 규칙**이다.

AI 에이전트 개발 패러다임 변화

① Prompt Engineering

모델에게 무엇을 말할까?

- 지시문 작성
- 예시 넣기
- 출력 형식 통제

한 번의 요청



② Context Engineering

어떤 정보를 언제 얼마나 줄까?

- 문맥 선택
- 검색 결과
- 메모리
- 토큰 예산

하나의 세션



③ Harness Engineering

에이전트를 어떤 환경에서 굴릴까?

- 도구 · MCP
- Agent Skills
- 메모리 계층
- 검증 루프
- 재사용 SOP

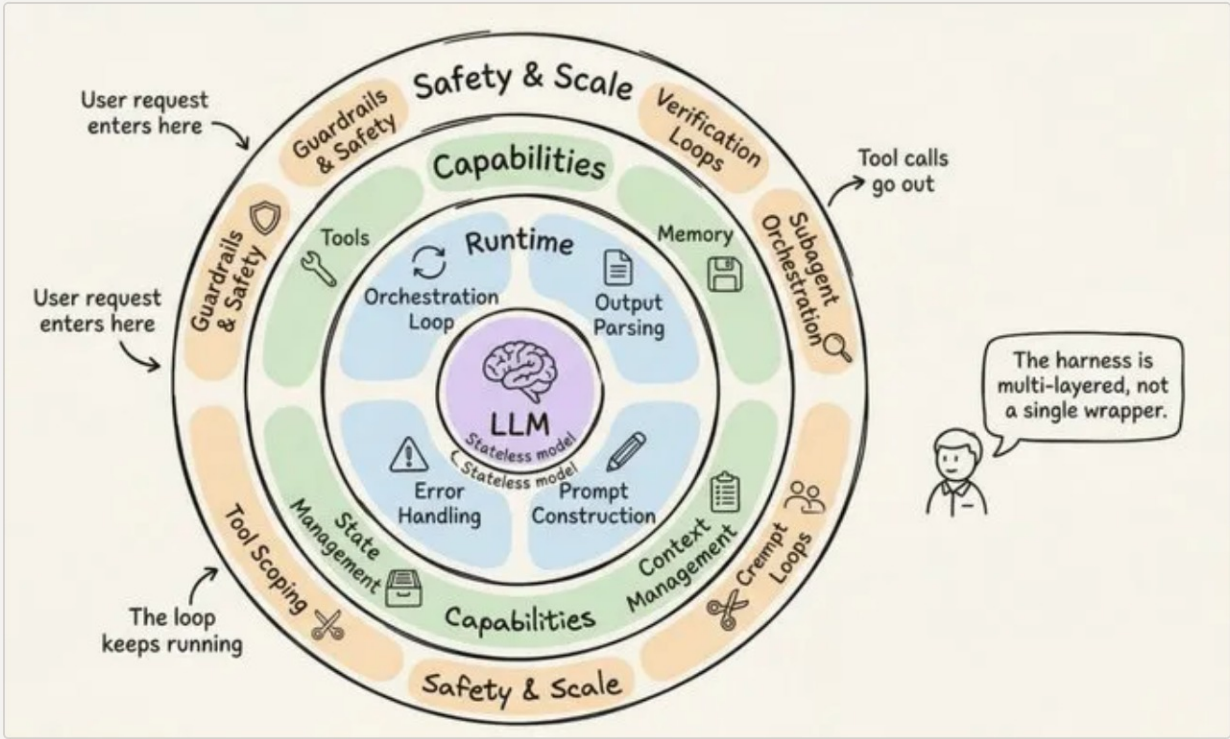
저장소 전체 · 팀 공유

FOCUS 경쟁력의 초점이 **프롬프트 작성** → **문맥 설계** → **실행 시스템 설계**로 이동

WHY 세 단계 모두 **LLM의 약점을 줄이기 위한 발전**이다

환각 · 맥락 망각 · 낡은 지식 · 검증 없는 완료 선언

하네스란 무엇인가



하네스는 LLM 이외의 모든 것이다

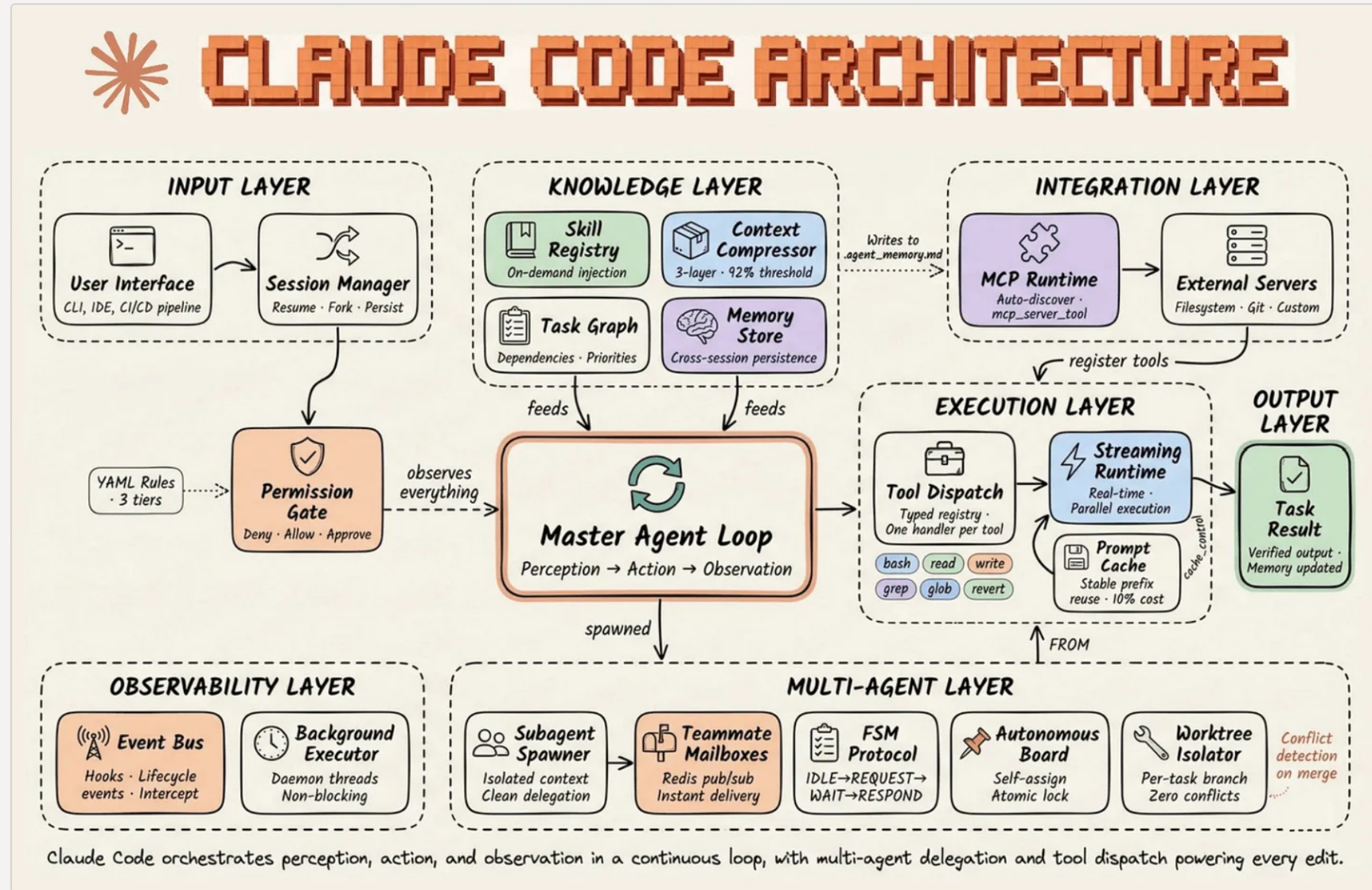
하나의 wrapper가 아니라 **multi-layered** 구조다.
모델은 추론만 하고, 나머지는 전부 이 layer들에서 일어난다.

RUNTIME	오케스트레이션 루프 · 출력 파싱 · 프롬프트 구성 · 에러 처리
CAPABILITIES	도구 · 메모리 · 상태 관리 · 컨텍스트 관리
SAFETY & SCALE	가드레일 · 검증 루프 · 도구 범위 제한 · 서브에이전트 조율

- 사용자 요청은 가장 바깥에서 들어온다
- ← 도구 호출은 가장 바깥으로 나간다
- ⌚ 그 사이에서 루프는 계속 돈다

모델은 고르는 것, 하네스는 만드는 것.

Agent = Model + Harness
도식 출처: 원저자 표기 필요



CORE

Master Agent Loop

인지 → 행동 → 관찰을 반복한다. 나머지 전부가 이 루프를 돕는 장치다.

GATE

Permission Gate

모든 실행이 거쳐가는 관문. 차단 · 허용 · 승인 요청으로 갈린다.

FEED

Knowledge Layer

스킬 · 메모리 · 컨텍스트 압축. 루프에 무엇을 쥐어줄지 정한다.

ACT

Execution Layer

도구 디스패치. bash · read · write · grep이 실제 변경을 만든다.

WATCH

Observability · Multi-Agent

혹이 지켜보고, 서브에이전트가 격리된 컨텍스트에서 일을 나눠 받는다.

설치하고, 물어보고, 확인한다

```
Windows PowerShell

# 1 · 설치 - PowerShell 에서
irm https://claude.ai/install.ps1 | iex
claude --version      새 창을 열고 확인

# 2 · 로그인 - 브라우저가 열린다
claude                Team 계정 선택 → 승인
/status              로그인 상태 확인
    다시 로그인하려면 /login

# 3 · 실습 프로젝트에서 실행
cd edu-stock-chart   교육 레포 안의 실습 프로젝트
claude

# 4 · 하네스 둘러보기
/model sonnet /permissions /context

# 5 · 첫 프롬프트
> 이 프로젝트를 분석해서 구조와 실행 방법을
  docs/analysis.md 로 정리해줘

# 6 · 다시 확인
/context              얼마나 찾나
```

- 01 설치 · 로그인
설치 후 새 창을 열어야 PATH가 잡힌다. 첫 claude 실행에서 브라우저로 Team 계정 로그인
- 02 프로젝트 안에서 연다
열린 디렉토리가 곧 작업 범위. 경로에 공백·한글이 없는 곳이 안전하다
- 03 하네스를 눈으로 본다
실습은 Sonnet으로 통일. 세 명령 모두 모델을 호출하지 않아 토큰을 쓰지 않는다
- 04 한 문장으로 한 바퀴
탐색 → 정리 → 파일 쓰기 권한 요청 → 결과물
- 05 전후를 비교한다
대화과 읽은 파일이 컨텍스트의 대부분을 차지한다

관찰 포인트

- 어떤 파일을 어떤 순서로 여는가 - 아무도 지시하지 않았다
- md 파일을 쓰기 직전에 멈춰서 묻는가 - Permission Gate
- /context 가 전후로 얼마나 달라졌는가

※ Git for Windows 설치 권장 - 있으면 Bash 도구, 없으면 PowerShell 도구로 동작
※ 대안 설치 winget install Anthropic.ClaudeCode · CMD는 install.cmd 방식

매일 쓰는 명령

1 순위 컨텍스트를 관리한다

<code>/context</code>	지금 무엇이 얼마나 들어있는지 본다
<code>/clear</code>	대화를 비우고 새로 시작한다 주제가 바뀌면 무조건
<code>/compact</code>	요약해 공간을 확보한다 <code>/compact</code> API 변경 중심으로
<code>/model</code>	모델을 바꾼다 작업 난이도에 맞춰
<code>/usage</code> · <code>/cost</code>	사용량과 비용을 본다

2 순위 세션을 다룬다

<code>/diff</code>	AI가 무엇을 고쳤는지 본다
<code>/rewind</code>	체크포인트로 코드·대화를 되돌린다
<code>/resume</code>	이전 대화를 이어간다
<code>/branch</code>	현재 지점에서 대화를 분기한다
<code>/btw</code>	기록에 남기지 않고 물어본다 컨텍스트를 늘리지 않는다
<code>/status</code>	계정 · 모델 · 프로젝트 상태

주제가 바뀌면 `/clear`. 이 하나만 습관이 되어도 절반은 된다.

하네스를 만드는 명령

3순위 구성 요소를 만든다

/init	프로젝트를 읽고 CLAUDE.md를 만든다 RECIPE 02
/memory	메모리 파일을 직접 편집한다 RECIPE 02
/permissions	허용 · 차단 규칙을 정한다 RECIPE 02
/hooks	훅을 확인하고 등록한다 RECIPE 03
/skill-creator: skill-creator	나만의 스킬을 만든다 공식 플러그인 · 설치 필요

4순위 도구를 붙이고 품질을 본다

/mcp	외부 도구 서버를 연결한다 RECIPE 04
/plugin	플러그인을 설치하고 배포한다 RECIPE 04
/code-review	코드를 검토한다 low ~ max · --fix 로 바로 수정
/security-review	보안 취약점을 검토한다
/verify	실제로 앱을 띄워 동작을 확인한다
/doctor	설정을 진단하고 고친다

오른쪽의 **RECIPE 번호**가 앞으로의 실습 순서다. 하나씩 만들어 내 하네스가 된다.

손이 먼저 기억해야 할 키

필수 **이 셋만은 외운다**

Shift + Tab	권한 모드를 순환한다 Plan 모드 진입이 여기
Esc	즉시 중단한다 컨텍스트는 유지된다
Esc Esc	되감기 메뉴 = /rewind
Ctrl + R	명령 기록 검색

SHIFT+TAB **권한 모드 네 가지**

AUTO	분류기가 위험한 것만 막는다 · 기본값
MANUAL	편집과 명령마다 사람이 승인
ACCEPT EDITS	파일 편집은 자동, 그 외는 확인
PLAN	소스를 건드리지 않고 계획만 세운다

사람이 개입하는 두 지점이 **이 두 키**에 있다 — Plan으로 들어가고, Esc로 멈춘다.

개발에 유용한 플러그인

A 계획 · 품질 프로세스

superpowers

권장

계획 → TDD → 디버깅 프로세스

feature-dev

탐색 → 설계 → 구현 워크플로

code-simplifier

코드를 읽기 좋게 정리

commit-commands

커밋 · 푸시 · PR 자동화

hookify

막고 싶은 동작을 흑으로

claude-md-management

CLAUDE.md 유지 · 개선

aidlc

권장

AI 주도 개발 라이프사이클 (AI-DLC)
AWS 오픈소스 · 공식 마켓 아님

agent-skills

권장

스펙 → 계획 → 구현 → 테스트 → 리뷰
Addy Osmani · 공식 마켓 아님

B 코드 품질 · 안전

jdtls-lsp

Java 오류 진단 · 심볼 탐색 · 바이너리 별도 설치

typescript-lsp

TS · JS 오류 진단 · npm 설치 필요

security-guidance

보안 패턴 경고

claude-security

내 코드 심층 취약점 스캔

code-modernization

레거시 코드 현대화 가이드

frontend-design

화면 디자인 가이드 (UI)

ponytail

실습

가장 단순한 해법을 먼저 고른다
공식 마켓 아님

C 외부 연동 · 메모리 (MCP)

context7

실습

최신 라이브러리 문서 조회

playwright

실습

브라우저로 화면 검증

github

공식 GitHub 연동

atlassian

소개만

Jira · Confluence 연동

claude-mem

실습

세션 기록을 저장해 다음 세션에 주입
오픈소스 · 공식 마켓 아님

D 문서화 · 시각화

eli5

실습

코드 · 오류를 눈높이에 맞춰 설명
공식 마켓 아님

archify

실습

아키텍처 · 시퀀스 도식을 HTML로
공식 마켓 아님

remotion

실습

React로 영상 만들기
공식 마켓 아님

graphify

코드 · 문서를 지식 그래프로 · HTML 시각화
오픈소스 · 공식 마켓 아님

설치는 공식 마켓에서. 플러그인은 흑과 MCP를 실행할 수 있으니 서드파티는 검토 후에.

실습

설치 실습 대상

권장

권장

찾고, 설치하고, 확인한다

● ● ● claude

1 · 찾기

`/plugin` Discover 탭에서 검색

2 · 설치

`/plugin install context7@claude-plugins-official`

3 · 활성화 · 확인

`/reload-plugins` "Active now"면 생략
`/` /mcp 에서 context7 이 보이면 성공
`claude plugin list` 셸에서 목록 · 상태

4 · 공식이 아닌 마켓에서 (remotion · ponytail · claude-mem)

`/plugin marketplace add remotion-dev/claude-code-plugin`
`/plugin install remotion@remotion`
`/plugin marketplace add DietrichGebert/ponytail`
`/plugin install ponytail@ponytail`
`/plugin marketplace add thedotmack/claude-mem`
`/plugin install claude-mem`

5 · 스킬만 있는 저장소 (플러그인 아님)

`npm skills add tt-ali/archify -g`
`git clone https://github.com/Dreambig0u/ELI5.git`
`cp -r ELI5/skills/eli5 ~/.claude/skills/eli5`

01 설치 전에 상세창을 읽는다

Will install — 명령 · 에이전트 · 스킬 · **혹** · MCP · LSP

02 범위 세 가지 중 하나

나만 · 이 저장소 팀 전체 · **나만, 이 저장소에서만**

03 공식 마켓은 처음부터 있다

첫 실행 때 자동 등록. 없으면
`/plugin marketplace add anthropics/claude-plugins-official`

04 켜고 끄고 지운다

`/plugin` → Installed 탭 · `claude plugin disable` · `uninstall`

주의

- 플러그인은 **혹과 MCP 서버**를 실행할 수 있다 — 상세창을 읽고 설치한다
- 공식 마켓 밖은 **Anthropic이 검토하지 않는다**

플러그인 일곱 개를 설치한다

	플러그인	설치 · 클릭하면 복사	확인
01	context7 최신 문서 조회 (MCP)	<code>/plugin install context7@claude-plugins-official</code>	"Spring Boot 3.4 문서를 조회해줘" → 도구 호출이 뜨는지
02	playwright 브라우저로 화면 검증 (MCP)	<code>/plugin install playwright@claude-plugins-official</code>	"localhost:5173 을 열어줘" → 브라우저 도구 호출이 뜨는지
03	ponytail 가장 단순한 해법을 먼저 고른다	<code>/plugin marketplace add DietrichGebert/ponytail</code> <code>/plugin install ponytail@ponytail</code>	새 세션에서 답변이 짧아지고 단순해지는지
04	remotion React로 영상 만들기	<code>/plugin marketplace add remotion-dev/claude-code-plugin</code> <code>/plugin install remotion@remotion</code>	<code>/</code> 를 눌러 remotion 항목이 보이는지
05	archify 아키텍처 · 시퀀스 도식	<code>npx skills add tt-ai/archify -g</code>	<code>/</code> 를 눌러 archify 스킬이 보이는지
06	eli5 코드 · 오류를 눈높이에 맞춰 설명	<code>\$ git clone https://github.com/Dreambig0u/ELI5.git</code> <code>\$ cp -r ELI5/skills/eli5 ~/.claude/skills/eli5</code>	"이 오류를 초등학생에게 설명해줘" → eli5 스킬이 쓰이는지
07	claude-mem 세션 기억 저장 · 주입	<code>/plugin marketplace add thedotmack/claude-mem</code> <code>/plugin install claude-mem</code>	Claude Code 재시작 후 <code>/mcp</code> 에 claude-mem 이 보이는지

범위

세 번째 "이 저장소에서만" 을 고른다 — 실습이 끝나면 개인 환경에 남지 않는다

정리

```
claude plugin uninstall <이름>@claude-plugins-official --scope local
```

개발 사이클을 이끄는 두 가지 스킬 모음



superpowers★ 293,282

Jesse Vincent · MIT · github.com/obra/superpowers

자동으로 발동하는 개발 방법론

코드 전에 질문하고, 계획 · TDD · 리뷰를 스킬이 강제한다



agent-skills★ 100,054

Addy Osmani · MIT · github.com/addyosmani/agent-skills

명령으로 운전하는 개발 라이프사이클

/spec → /plan → /build → /test → /review → /ship

superpowers · 자동으로 발동			agent-skills · 명령으로 실행		
01	요구사항·스펙	brainstorming 질문으로 설계를 다듬고 조각별로 승인	interview-me 한 번에 한 질문으로 요구사항 정리	/spec	코드 전에 구조화된 스펙
02	계획	writing-plans 2~5분짜리 태스크 · 파일 경로 · 검증 단계 using-git-worktrees 격리된 브랜치에서 작업	/plan 작고 원자적인 태스크	/constraints	품질 기준을 CONSTRAINTS.md에
03	구현	subagent-driven-development 태스크마다 서브에이전트가 구현 · 검토	/build	한 조각씩 구현 · 테스트 · 커밋	
04	실행·검증	test-driven-development RED → GREEN → REFACTOR 강제 systematic-debugging 4단계 원인 추적	/test	TDD · 버그는 재현 테스트부터	
05	리뷰	requesting-code-review 태스크 사이마다 심각도별 점검	/review	5축 리뷰 (정확성·가독성·구조·보안·성능)	
			/code-simplify	동작은 그대로, 복잡도만 줄임	
06	마무리	finishing-a-development-branch merge · PR · 유지 · 폐기 선택	/ship	병렬 점검 후 go / no-go	

개발 구현에 활용하는 플러그인

ponytail

실습

★ 148,682

LLM의 약점

요청보다 크게 만든다 — 라이브러리 설치, 래퍼 컴포넌트, 불필요한 추상화로 코드가 장황해진다

해법

"가장 게으른 시니어 개발자"처럼 안 만들기 → 있는 것 재사용 → 표준·네이티브 기능 → 한 줄 순으로 가장 단순한 해법을 먼저 고른다

핵심 기능

사다리 필요한가? → 코드베이스에 있나? → 표준 라이브러리 → 네이티브

강도

lite · full · ultra 선택

/ponytail-review · audit

과잉 설계를 찾아 삭제 제안

예) 날짜 선택기 404줄 → 23줄 (<input type="date">)

코드 -54% · 토큰 -22% · 비용 -20% · 시간 -27%

README 자체 측정 · 기능 12개 · Haiku 4.5

context7

실습

★ 62,556

LLM의 약점

학습 시점에 멈춘 지식 — 옛 버전 답, 존재하지 않는 API를 지어낸다(환각)

해법

공식 문서와 예제를 버전별로 실시간 조회해 프롬프트에 넣는다. 모델이 기억이 아니라 최신 문서를 보고 코드를 쓴다

핵심 기능

use context7 를 붙이거나 자동 발동 규칙으로 사용

MCP 도구 2개

resolve-library-id → query-docs

버전 · ID 지정

"Next.js 14" · /vercel/next.js

예) "Spring Boot 3.4 문서를 조회해줘" → 도구 호출이 화면에 뜬다

무료 API 키를 발급하면 호출 한도가 올라간다 (권장)

claude-mem

실습

★ 94,993

LLM의 약점

세션이 끝나면 기억이 사라진다 — 매번 프로젝트 코드를 다시 읽어 파악하느라 토큰이 많이 든다

해법

세션 중 한 일을 자동으로 기록 · 요약해 로컬에 저장하고, 다음 세션 시작 때 필요한 맥락을 자동으로 넣어 준다

핵심 기능

자동 캡처 혹은 도구 사용을 기록 → 로컬 SQLite

3단계 검색

search(목록) → timeline → get_observations(필요한 것만)

<private>

태그로 저장 제외 · 웹 뷰어

3단계 검색으로 상세 조회를 줄여 토큰 약 10배 절감 (README 주장)

※ 기록 · 요약 자체에도 소량의 토큰이 든다

플러그인 세 개를 써 본다

먼저 실습 프로젝트 폴더에서 Claude Code 를 연다 `cd edu-stock-chart` `claude` · claude-mem 은 설치 뒤 Claude Code 재시작

ponytail

강사 시연

과잉 설계를 찾아 **지울 목록**만 받는다

01 전체 점검

`/ponytail-audit`

저장소 전체를 훑어 삭제 · 단순화 후보를 순위로 보여 준다 (수정은 하지 않는다)

02 하나만 적용

가장 효과 큰 1개만 적용하고 테스트를 돌려줘

제안을 그대로 다 받지 말고 한 건만 골라 적용한다

볼 것 항목마다 「위치 · 자를 것 · 대체 방법」이 한 줄로 나오는가

context7

실습 1

최신 문서를 근거로 Spring Boot 스펙을 묻는다

01 질문 끝에 use context7

Spring Boot 3.4에서 `spring.jpa.hibernate.ddl-auto` 값과 기본값을 알려줘. `use context7`

프롬프트에 `use context7` 을 붙이면 문서 조회를 거친다

02 비교 (선택)

Spring Boot 3.4에서 `spring.jpa.hibernate.ddl-auto` 값과 기본값을 알려줘

같은 질문을 `use context7` 없이 물어 답을 비교한다

확인 화면에 `resolve-library-id` → `query-docs` 도구 호출이 뜨는가

claude-mem

실습 2

프로젝트 전체를 기억에 미리 넣는다

01 전체 소스를 읽혀 기억시키기

`/learn-codebase`

모든 소스 파일을 읽어 기억에 저장한다 (약 5분 · 토큰 소모 큼). 끝나면 `/exit`

확인 새 세션에서 프로젝트를 물었을 때 코드를 다시 읽지 않고 답하는가 · `/mcp` 에 `claude-mem` 이 보이는가

코드는 리뷰로, 화면은 브라우저로 검사한다

code-review 기본 제공

공식 플러그인도 있음

변경한 코드를 여러 관점으로 검토해 진짜 문제만 골라 준다

두 가지 방식

기본 명령 `/code-review` 현재 변경분(diff)이나 PR 번호를 검토. 강도 low ~ max 선택, `--fix` 로 바로 수정

공식 플러그인 `code-review@claude-plugins-official` 병렬 에이전트 4개가 PR을 검토 — CLAUDE.md 준수 2 · 버그 1 · 변경 이력(git blame) 1

오탐 줄이기 이슈마다 신뢰도 0~100 점수, **80 미만은 제외**하고 GitHub PR에 댓글로 게시

함께 쓰는 명령

`/security-review` 보안 점검 · `/simplify` 재사용 · 단순화 정리

`/code-review`

주의 PR 리뷰는 GitHub 원격과 `gh` 가 필요하다. `ultra` 강도는 클라우드에서 돌고 별도 과금된다

playwright 실습

★ 37,713

브라우저를 직접 열어 만든 화면을 눌러 보고 확인한다

어떻게 동작하나

Playwright MCP (Microsoft) — 페이지의 **접근성 트리**(구조화된 텍스트)를 읽어 조작. 스크린샷 · 비전 모델이 필요 없다

도구 `browser_navigate` · `browser_snapshot` · `browser_click` · `browser_fill_form` · `browser_take_screenshot`

품질 확인 콘솔 오류(`browser_console_messages`) · 네트워크 실패 (`browser_network_requests`)까지 본다

이런 때 쓴다

구현한 UI가 실제로 열리고 차트가 그려지는지 · 폼 입력 흐름 · 화면 오류 재현

`localhost:5173` 을 열어서 차트가 그려지는지, 콘솔 오류가 없는지 확인해줘

주의 도구 정의와 접근성 트리가 **토큰을 많이 쓴다**. 저장소도 토큰 효율은 CLI + 스킴 방식이 낫다고 안내한다 (Node 18 이상)

개발 산출물 · 코드 분석 및 질의

eli5 **실습**

★ 1,567

5세

초등 5학년

대학생

매니저

엔지니어

디자이너

부모

배우자

누구에게 설명하느냐에 맞춰 답을 바꿔 준다

제공

대상 4종류: 나이 · 학년 · 직무 · 관계

어휘 · 비유 · 어조 · 깊이 · 관점을 대상에 맞춰 조정

장점

코드 · 오류 · 개념을 비개발자에게 바로 설명

스킬 파일 하나라 설치가 가볍다

이 오류를 초등학생에게 설명해줘

archify **실습**

★ 74,917



코드를 읽고 다이어그램을 HTML 한 파일로 만든다

제공

아키텍처 · 워크플로 · 시퀀스 · 데이터 플로우 · 라이프사이클 5종

다크/라이트 · 경로 추적 · PNG · SVG · 영상 내보내기

장점

저장소를 읽어 소스 근거의 다이어그램

채팅으로 계속 수정 (“Redis 추가”)

HTML 파일이라 설치 없이 공유

edu-stock-chart의 주식 조회 흐름을 archify로 시퀀스 다이어그램으로 그려줘

remotion **실습**

★ 61,240



React 코드로 영상을 만든다 — 코드가 원본

제공

스킬 remotion-create · studio(미리보기) · render(내보내기)

웹앱에 넣는 Player · 대량 렌더링

활용

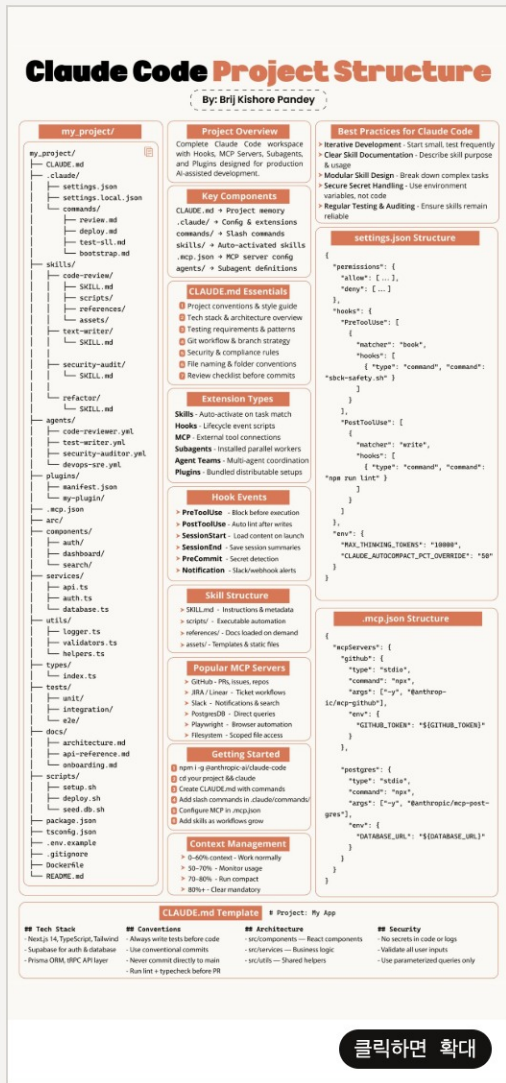
발표 · 소개 · 데이터 연동 영상

에이전트에게 말로 시키고 코드로 다듬기

수정 · 재사용 · 버전 관리가 쉽다

remotion으로 edu-stock-chart를 소개하는 15초 영상을 만들어줘

Claude Code 프로젝트 구조



CLAUDE.md

프로젝트 메모리. 세션마다 **항상 읽힌다** (다음 장)

.claude/settings.local.json

내 로컬 설정. git에 올리지 않는다

skills/<이름>/SKILL.md

작업이 맞으면 **자동 활성화**되는 스킬. scripts · references · assets 포함

.mcp.json

외부 도구(MCP 서버) 연결: GitHub · DB · Playwright

Hooks

특정 시점에 스크립트 실행. 실행 전 차단 · 수정 후 자동 lint

.claude/settings.json

팀 공유 설정. permissions · hooks · env

.claude/commands/

슬래시 명령. md 파일 하나가 명령 하나 (/review)

agents/

서브에이전트 정의 (code-reviewer 등). 역할을 나눠 병렬로 일한다

plugins/

스킬 · 명령 · 에이전트 · 훅 · MCP를 묶어 **배포**하는 단위

components/ · services/ · tests/ · docs/ ...

일반 앱 코드와 문서. 그림의 예시 프로젝트 구조

그림은 서드파티 인포그래픽 (Brij Kishore Pandey). 공식 문서와 다른 곳: skills/ · agents/ 는 .claude/ 아래에 둔다 · 에이전트 파일은 .md · PreCommit 은 공식 혹은 이벤트가 아니다

매 번 읽 히 는 프 로젝 트 배 경 지 식

CLAUDE.md

역 할

세션이 시작될 때마다 **자동으로 읽혀** 컨텍스트에 들어간다

그래서 **반복해서 주입하는 프롬프트 · 배경지식**을 적는다. 대상은 **자주 바뀌지 않는 정보**

어 디 에 두 나 · 읽 히 는 순 서		
01	조직 정책 파일	그 PC의 모든 사용자 · 개인이 제외할 수 없다
02	~/claude/CLAUDE.md	내 모든 프로젝트 · 나만
03	./CLAUDE.md	이 프로젝트 · 팀과 공유 (git)
04	./CLAUDE.local.md	이 프로젝트 · 나만 (.gitignore)
05	하위 폴더 CLAUDE.md	그 폴더의 파일을 읽을 때만 로드

덮어쓰지 않고 이어 붙인다 · 실행 위치에 가까운 파일이 나중에 읽힌다 · 충돌 시 따를 쪽이 보장되지 않으니 **모순되게 쓰지 않는다**

C L A U D E . m d 에 넣 는 정 보

기술 스택 Spring Boot 3.4 · Java 21 Gradle · JPA + H2	명령어 gradlew.bat build gradlew.bat test · bootRun	프로젝트 구성 패키지 구조 컨트롤러 → 서비스 → 리포지토리	Git 규칙 브랜치 전략 커밋 메시지 규칙	프로젝트 설명 무엇을 위한 프로젝트인가 주식 차트 연습용 웹 서비스
---	--	---	---	---

좋은 CLAUDE.md 쓰는 법

예시 · 실습 프로젝트의 CLAUDE.md · 태그 = 들어가는 분류

```
# edu-stock-chart
주식 시세를 조회해 차트로 보여 주는 교육용 웹 서비스 프로젝트 설명

## Tech Stack 기술 스택
- Spring Boot 3.4, Java 21, Gradle 8.6, JPA + H2, Lombok
- UI: Vite 6 + Chart.js 4 (edu-stock-chart-ui)

## Commands 명령어
- `gradlew.bat build` - 빌드 + 테스트
- `gradlew.bat test` - 단위 테스트
- `gradlew.bat bootRun` - 실행 (http://localhost:8080)

## Architecture 프로젝트 구성
- 패키지: `kr.co.koscom.edu.stockchart`
- 컨트롤러 → 서비스 → 리포지토리 (로직은 서비스에)

## Git Git 규칙
- 브랜치: `feature/*` → `main`
- 커밋 메시지: 한글, `type: 요약`
```

핵심 5가지 · HumanLayer

- 01 **꼭 알려줄 것만**
무엇으로 만들었나(스택 · 구조) · 왜 만드나(목적) · 어떻게 일하나(명령 · 검증)
- 02 **모든 작업에 해당하는 것만**
일부 작업에만 맞는 지시는 뻔다. 관련 없는 내용이 많을수록 무시되기 쉽다
- 03 **점진적 공개**
세부 내용은 별도 md로 나누고, CLAUDE.md에는 어디를 보면 되는지만 적는다. 코드 복사 대신 file:line
- 04 **스타일은 린터에게**
포매터 · 린터(필요하면 hook)에 맡긴다. Claude는 린터가 아니다
- 05 **자동 생성은 초안으로만**
/init 결과를 그대로 쓰지 말고 한 줄씩 검토한다. 모든 단계에 영향을 주는 파일이다

주의 사항 내용이 많다고 무조건 좋은 것이 아니다. 지시가 늘수록 뒤쪽만이 아니라 **전부 고르게 덜 지켜진다**

모델 크기별 권장 지시 수	대형 150 ~ 200	중형 80 ~ 120	소형 40 ~ 60
----------------	--------------	-------------	------------

꿀 팁 CLAUDE.md는 **항상 컨텍스트에 들어가 토큰을 쓴다**. 정말 필요한 것만 쓰고, **상황별 내용은 별도 md 파일로 빼서** CLAUDE.md에는 **경로와 언제 읽는지만** 적는다.
DB 스키마를 바꿀 때 → docs/db-schema.md (@import 는 시작 시 함께 로드되어 절약되지 않는다)

CLAUDE.md를 만들어 본다

단계	입력 · 클릭하면 복사	확인할 것
01 준비 프로젝트 폴더에서 Claude Code 실행	cd edu-stock-chart claude	프롬프트 입력창이 열린다
02 초안 만들기 /init 이 코드를 분석해 CLAUDE.md 를 만든다	/init	edu-stock-chart/CLAUDE.md 가 생긴다. 파일이 이미 있으면 덮어쓰지 않고 개선안을 제안한다
03 정리하기 초안에서 필요한 것만 남긴다	CLAUDE.md를 정리해줘. 기술 스택, 빌드 · 테스트 · 실행 명령, 패키지 구성과 계층 규칙, Git 규칙, 프로젝트 설명만 남기고 코드를 읽으면 알 수 있는 내용과 파일별 설명은 지워줘. 100줄 이하로.	한 줄씩 읽는다. 빠도 실수하지 않을 줄은 지운다
04 규칙 추가 우리 팀 규칙을 한두 줄 넣는다	CLAUDE.md의 Git 섹션에 규칙을 추가해줘. 브랜치는 feature/* 에서 작업해 main 으로 PR 머지하고, 커밋 메시지는 한글로 "type: 요약" 형식으로 쓴다.	Git 섹션에 두 줄이 늘었는가 (규칙은 예시이니 팀 기준으로 바꾼다)
05 로드 확인 CLAUDE.md 가 읽히는지 본다	/context	목록의 Memory files 아래에 CLAUDE.md 가 보이는가
06 새 세션 검증 기억 없이 시작해도 아는지 묻는다	/exit claude 이 프로젝트 테스트는 어떻게 실행해?	코드를 다시 읽지 않고 CLAUDE.md 의 명령으로 답하는가

초안은 그대로 쓰지 않는다. CLAUDE.md는 매 세션 들어가므로 정리 단계가 핵심이다 입력이 여러 줄이면 위에서부터 차례로 입력한다